Colonies de fourmis et problème du voyageur de commerce

Précision contre temps de calcul : quand utiliser l'ACO ?

Ivano Balboni Quentin Wache

Novembre 2026

Plan

1. Le problème et les méthodes classiques
2. Le modèle ACO
3. Ce que prédit la théorie
4. Protocole expérimental
5. Complexité : mesure vs théorie
6. Précision vs temps de calcul
7. Cas d'usage et limites

Question centrale : pour un temps de calcul donné, quel algorithme donne la meilleure tournée, et l'ACO a-t-il sa place ?

Le problème

Le voyageur de commerce (TSP)

- n villes, distances d_{ij}
- Trouver le cycle le plus court qui passe une fois par chaque ville
- Problème NP-difficile : aucun algorithme polynomial connu
- Nombre de tournées distinctes : $\dfrac{(n-1)!}{2}$

n	tournées possibles
10	$1{,}8 \times 10^{5}$
20	$6{,}1 \times 10^{16}$
30	$4{,}4 \times 10^{30}$
50	$3{,}0 \times 10^{62}$
100	$4{,}7 \times 10^{155}$

À un milliard de tournées par seconde, $n = 20$ prend déjà 2 ans. Il faut donc choisir entre exactitude et temps de calcul.

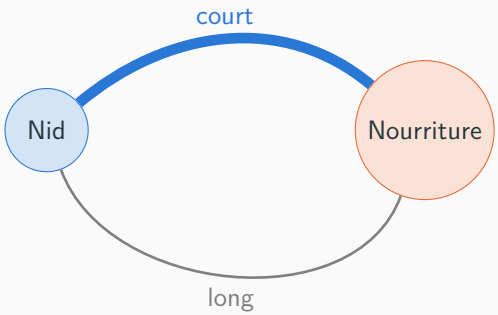
Les algorithmes comparés

Algorithme	Famille	Complexité théorique	Garantie
Held-Karp	exact (prog. dynamique)	$O(n^2 2^n)$	optimum
Plus proche voisin (PPV)	glouton	$O(n^2)$	aucune
PPV + 2-opt	recherche locale	$O(n^2)$ par passe	optimum local
Christofides	approximation	$O(n^3)$	$\leq 1{,}5\times$ optimum
Recuit simulé	métaheuristique	O(itérations)	asymptotique
OR-Tools (Google)	solveur industriel	limite de temps	aucune
ACO (AS, ACS, MMAS)	métaheuristique	$O(T\,m\,n^2)$	asymptotique
ACO MMAS + 2-opt	hybride	$O(T\,m\,n^2)$ + 2-opt	asymptotique

T : itérations, m : fourmis. Témoin ajouté : 2-opt multi-départs (2-opt relancé depuis des tournées aléatoires), pour isoler l'apport de l'apprentissage.

Le modèle ACO

Inspiration : l'expérience du double pont



- Les fourmis déposent une phéromone sur leur chemin
- Sur le chemin court, les allers-retours sont plus rapides ⇒ plus de dépôt
- Plus de phéromone ⇒ plus de fourmis : rétroaction positive
- L'évaporation efface les mauvais chemins

Deneubourg et al. (1990) ; communication indirecte par l'environnement = *stigmergie*

Ant System (Dorigo et al., 1996)

À chaque itération, m fourmis construisent chacune une tournée.

1. Choix de la ville suivante (fourmi k en i, villes non visitées $\mathcal{N}_k$) :

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\, \eta_{ij}^{\beta}}{\sum_{l \in \mathcal{N}_k} \tau_{il}^{\alpha}\, \eta_{il}^{\beta}} \qquad \eta_{ij} = \frac{1}{d_{ij}}$$

2. Mise à jour des phéromones (après toutes les fourmis) :

$$\tau_{ij} \leftarrow \underbrace{(1-\rho)\,\tau_{ij}}_{\text{évaporation}} + \underbrace{\sum_k \frac{Q}{L_k} \mathbb{1}[(i,j) \in T_k]}_{\text{dépôt}}$$

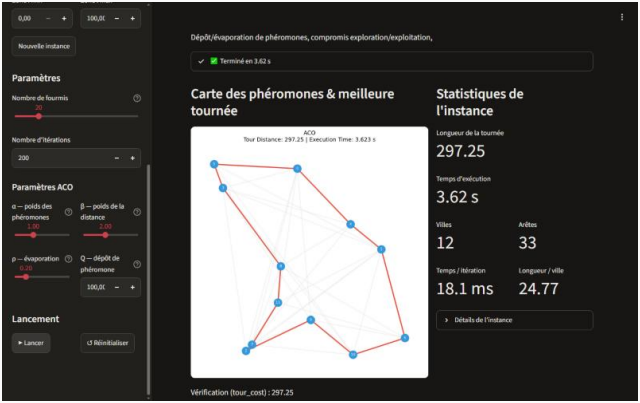
α : poids de la mémoire collective

β : poids de la distance (glouton)

ρ : vitesse d'oubli

m : nombre de fourmis, L_k : longueur de la tournée k

Démonstration



En direct, 20 villes :

1. Réglages de référence : $\alpha = 1$, $\beta = 2$, $\rho = 0{,}2$
2. $\alpha = 3$: la colonie se fige sur une piste trop tôt
3. $\beta = 0$: sans la distance, presque du hasard

`using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`

Variantes testées

AS Ant System : toutes les fourmis déposent

ACS Ant Colony System (Dorigo & Gambardella, 1997) : avec probabilité $q_0 = 0{,}9$, choix de la meilleure arête ; mise à jour locale pendant la construction

MMAS MAX-MIN Ant System (Stützle & Hoos, 2000) : seule la meilleure fourmi dépose, $\tau \in [\tau_{\min}, \tau_{\max}]$ avec $\tau_{\max} = \frac{1}{\rho L^*}$, réinitialisation si la colonie stagne

+ 2-opt Recherche locale appliquée aux meilleures tournées de chaque itération

Point commun
Ce sont des algorithmes anytime : on peut les arrêter à tout moment, et la solution s'améliore avec le temps accordé.

Ce que prédit la théorie

Complexité d'une itération d'ACO

- Construction : m fourmis $\times$ $(n-1)$ pas $\times$ $O(n)$ candidats

$$\Rightarrow O(m\,n^2)$$

- Évaporation $O(n^2)$, dépôt $O(m\,n)$
- Total pour T itérations : $\boxed{O(T\,m\,n^2)}$
- Mémoire : matrice des phéromones $O(n^2)$

Prédictions testables

- m et T fixés : temps $\propto n^2$ (pente 2 en log-log)
- Ajouter 2-opt : chaque passe coûte $O(n^2)$, et le nombre de passes croît avec n
- Held-Karp : temps $\times \approx 2$ par ville ajoutée

Dynamique des phéromones et convergence

Évaporation seule : $\tau_t = \tau_0(1-\rho)^t$

$\Rightarrow$ la colonie « oublie » en environ $1/\rho$ itérations $\quad (\rho = 0{,}2 : 5 \text{ it.} ; \rho = 0{,}02 : 50 \text{ it.})$

Dépôt constant Δ : $\tau_t \to \Delta/\rho$ (point fixe)

Résultats de convergence

- Gutjahr (2000) : un Ant System modifié trouve l'optimum avec une probabilité qui tend vers 1
- Stützle & Dorigo (2002) : avec $\tau_{\min} > 0$, la probabilité de trouver l'optimum tend vers 1

Mais

Garanties asymptotiques uniquement : aucun temps polynomial (le TSP est NP-difficile). La vraie question est donc expérimentale : quelle précision pour quel temps ?

Protocole expérimental

Protocole

Expérience 1 : complexité

- Instances aléatoires, $n = 25$ à 800
- Held-Karp : $n = 6$ à 16
- ACO : 25 fourmis, 10 itérations
- Pente mesurée en log-log sur les grands n

Expérience 2 : précision vs temps

- TSPLIB à optimum prouvé : eil51, kroA100, ch150
- Budget de 30 s, 3 graines par algorithme aléatoire
- On enregistre la meilleure tournée au cours du temps

Trois types d'algorithmes

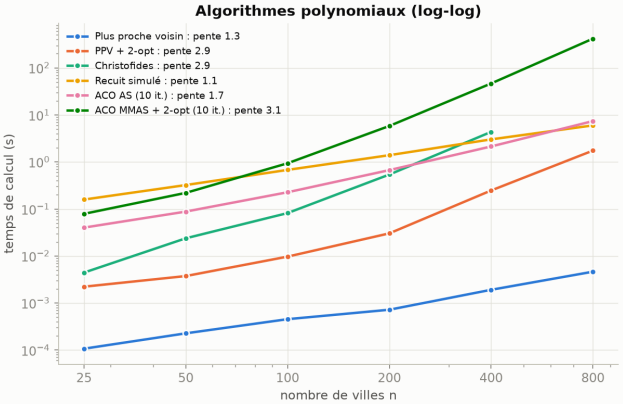
- *Un coup* (un point) : PPV, PPV + 2-opt, Christofides
- *Budget réglable* : recuit simulé (30 à 30 000 n itérations), OR-Tools (0,1 à 30 s)
- *Anytime* (une courbe) : ACO, 2-opt multi-départs

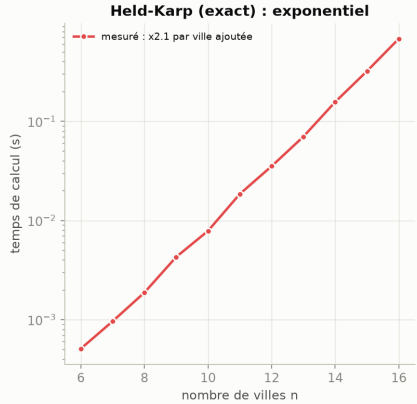
Équité

- Tous les algorithmes sur un seul cœur de la même machine
- Même budget de temps pour tous
- Paramètres standards de la littérature, sans réglage par instance

Complexité : mesure vs théorie

Temps de calcul en fonction de n





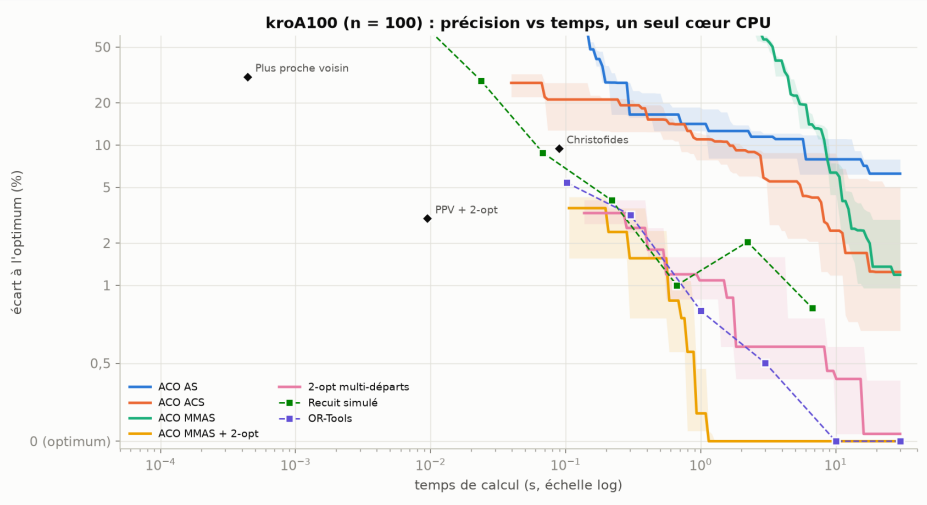
Pentes mesurées vs théorie

Algorithme	Théorie	Mesuré	Lecture
Christofides	3	**2,9**	conforme
PPV + 2-opt	≈ 3	**2,9**	$O(n^2)$ par passe $\times$ $O(n)$ passes
ACO AS (m, T fixés)	2	1,7	proche, coûts fixes par pas
ACO MMAS + 2-opt	> 2	3,1	dominé par le 2-opt
Plus proche voisin	2	1,3	boucle interne vectorisée
Recuit simulé	1 à 2	1,1	coût par itération quasi constant
Held-Karp	$\times$2,3 / ville	$\times$ **2,1**	exponentiel, conforme

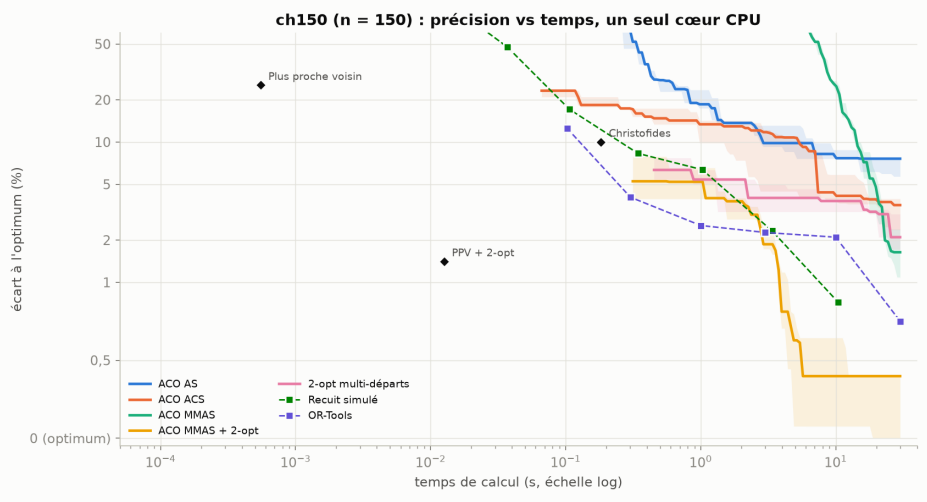
- Les écarts viennent des coûts fixes de l'implémentation : une opération sur n valeurs coûte presque autant pour $n = 25$ que pour $n = 800$. La théorie décrit le comportement pour n grand.
- Held-Karp : 0,7 s à $n = 16$; extrapolé à $n = 30$: $\approx$ 6 h et plus de 100 Go de mémoire.

Précision vs temps de calcul

kroA100 : précision en fonction du temps accordé



ch150 : précision en fonction du temps accordé



Quel algorithme gagne, selon le temps disponible ?

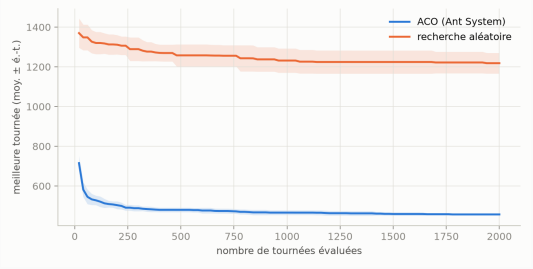
Budget	eil51 ($n = 51$)	kroA100 ($n = 100$)	ch150 ($n = 150$)
10 ms	PPV+2-opt (2,1 %)	PPV+2-opt (3,0 %)	PPV+2-opt* (1,4 %)
0,1 s	PPV+2-opt (2,1 %)	PPV+2-opt (3,0 %)	PPV+2-opt (1,4 %)
1 s	MMAS+2-opt (0,1 %)	MMAS+2-opt (0,2 %)	PPV+2-opt (1,4 %)
10 s	MMAS+2-opt (0,1 %)	MMAS+2-opt† (0 %)	MMAS+2-opt (0,4 %)
30 s	MMAS+2-opt‡ (0,1 %)	MMAS+2-opt† (0 %)	MMAS+2-opt (0,3 %)

* en 13 ms ; † ex æquo avec OR-Tools ; ‡ ex æquo avec 2-opt multi-départs.

- Temps très court : PPV + 2-opt est imbattable
- Dès que le budget permet plusieurs centaines d'itérations, MMAS + 2-opt prend la tête ; le seuil augmente avec n (< 1 s pour 100 villes, ≈ 4 s pour 150)
- Les ACO sans recherche locale ne gagnent jamais

L'apprentissage des fourmis sert-il vraiment ?

Contre le hasard (30 villes, 2 000 tournées)



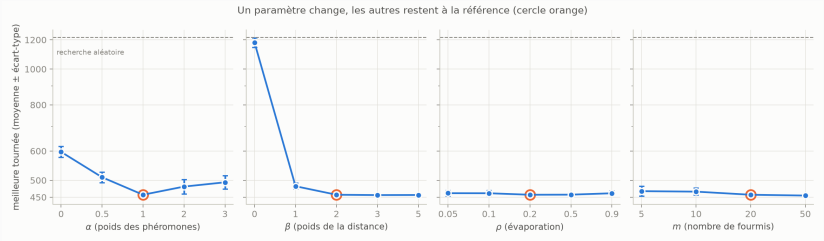
ACO : **457** ; recherche aléatoire : 1218

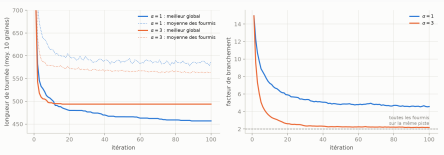
Contre le témoin 2-opt multi-départs

à 10 s (%)	2-opt m.-d.	MMAS+2-opt
eil51	0,55	**0,08**
kroA100	0,35	**0,00**
ch150	3,66	**0,37**

Même recherche locale, même temps : partir des tournées proposées par les fourmis plutôt que de tournées aléatoires divise l'écart par 7 à 10, et atteint l'optimum sur kroA100.

Paramètres : le compromis précision/temps dans l'ACO lui-même





- $\alpha = 3$: converge en ≈ 10 it. mais s'arrête à 494 ; $\alpha = 1$: ≈ 40 it., atteint 457
- $\beta = 0$: presque le hasard ; $\alpha = 0$ (pas de mémoire) : 597
- ρ : peu d'effet ; plus de fourmis : plus régulier (é.-t. 14,6 $\rightarrow$ 1,1)

30 villes, 10 graines. Détail : page « Analyse » de la démo.

Cas d'usage et limites

Quel algorithme pour quelle situation ? (TSP statique)

Situation	Choix	Justification mesurée
$n \leq 15$, optimum exigé	Held-Karp	exact en < 1 s ; $\times 2$ par ville ensuite
Réponse en quelques ms	PPV + 2-opt	1,4 à 3 % en 3 à 13 ms
Garantie théorique exigée	Christofides	$\leq 1{,}5\times$ prouvé ; mesuré 8 à 10 %
Quelques secondes, meilleure qualité	MMAS + 2-opt	0 à 0,4 % en 10 s ; optimum sur kroA100
Solveur prêt à l'emploi	OR-Tools	0 à 0,8 % en 30 s, sans réglage
ACO sans recherche locale	à éviter	0,4 à 7 % en 30 s ; toujours battu par une méthode avec 2-opt

L'ACO joue le rôle de générateur de bons points de départ pour la recherche locale.

Où l'ACO a un avantage structurel

Problèmes qui changent dans le temps

- Routage réseau (AntNet, Di Caro & Dorigo, 1998) : les phéromones s'adaptent quand les coûts changent, sans tout recalculer

Problèmes avec contraintes métier

- Tournées de véhicules (capacités, créneaux), ordonnancement d'ateliers, affectation quadratique
- La contrainte s'ajoute dans la règle de choix de la fourmi

Où ce n'est pas utile

Plus court chemin sur un graphe fixe : Dijkstra est exact en $O((V+E)\log V)$. Une métaheuristique ne se justifie que si le problème est NP-difficile ou change dans le temps.

Coût à prévoir

Réglage des paramètres (α, β, ρ) et quelques secondes de calcul avant de battre les heuristiques simples.

Limites de notre étude

- Trois instances TSPLIB, $n \leq 150$: le seuil où MMAS + 2-opt devient le meilleur choix augmente avec n et reste à mesurer au-delà
- Budget maximal de 30 s, sur une seule machine
- Paramètres standards de la littérature, non réglés par instance (ni pour l'ACO, ni pour le recuit simulé, ni pour OR-Tools)
- Implémentation Python : les temps absolus seraient plus faibles en C, mais les pentes et le classement devraient rester proches
- Les avantages « dynamique » et « contraintes » de l'ACO viennent de la littérature, pas de nos mesures

Conclusion

1. **La complexité mesurée suit la théorie** pour Christofides, 2-opt et Held-Karp ; ailleurs, les coûts fixes de l'implémentation dominent aux tailles testées
2. **Précision vs temps :** PPV + 2-opt domine sous la seconde ; avec quelques secondes, MMAS + 2-opt atteint l'optimum ou s'en approche à 0,3 %, au niveau d'OR-Tools
3. **L'apprentissage compte :** à recherche locale égale, partir des tournées des fourmis divise l'écart par 7 à 10 (et atteint l'optimum sur kroA100)
4. **Cas d'usage :** l'ACO seul ne vaut pas les heuristiques classiques sur le TSP statique ; son intérêt est d'être hybridé avec une recherche locale, ou utilisé sur des problèmes dynamiques et contraints

Merci ! Questions ? Démonstration : `using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`

Références

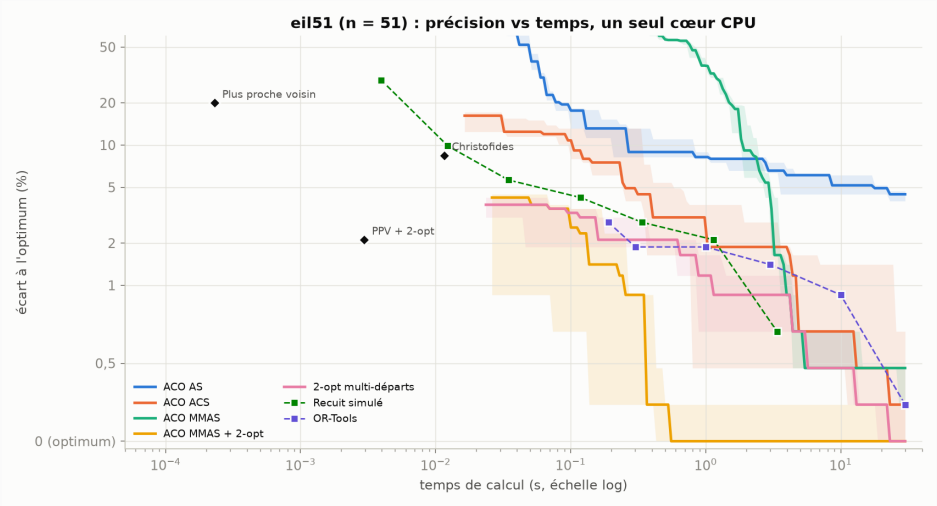
- M. Dorigo, V. Maniezzo, A. Colorni. *Ant System : optimization by a colony of cooperating agents*. IEEE Trans. SMC-B, 26(1), 1996.
- M. Dorigo, L. M. Gambardella. *Ant Colony System*. IEEE Trans. Evol. Comp., 1(1), 1997.
- T. Stützle, H. Hoos. *MAX-MIN Ant System*. Future Gen. Comp. Systems, 16(8), 2000.
- W. J. Gutjahr. *A graph-based Ant System and its convergence*. Future Gen. Comp. Systems, 16(8), 2000.
- T. Stützle, M. Dorigo. *A short convergence proof for a class of ACO algorithms*. IEEE Trans. Evol. Comp., 6(4), 2002.
- J.-L. Deneubourg, S. Aron, S. Goss, J. M. Pasteels. *The self-organizing exploratory pattern of the Argentine ant*. J. Insect Behavior, 3, 1990.
- G. Di Caro, M. Dorigo. *AntNet : distributed stigmergetic control for communications networks*. JAIR, 9, 1998.
- N. Christofides. *Worst-case analysis of a new heuristic for the travelling salesman problem*. Rapport, Carnegie-Mellon University, 1976.
- M. Held, R. M. Karp. *A dynamic programming approach to sequencing problems*. J. SIAM, 10(1), 1962.
- M. Dorigo, T. Stützle. *Ant Colony Optimization*. MIT Press, 2004.
- G. Reinelt. *TSPLIB*. ORSA J. Computing, 3(4), 1991.

Annexe : écart à l'optimum (%) pour un budget de 1 s et de 30 s

	budget 1 s			budget 30 s		
	eil51	kroA100	ch150	eil51	kroA100	ch150
Plus proche voisin	20,0	30,7	25,5	20,0	30,7	25,5
PPV + 2-opt	2,1	3,0	**1,4**	2,1	3,0	1,4
Christofides	8,5	9,5	10,0	8,5	9,5	10,0
Recuit simulé	2,5	2,0	6,3	1,5	0,8	0,9
OR-Tools	1,9	0,8	2,5	0,2	**0,0**	0,8
2-opt multi-départs	1,2	1,2	5,6	**0,1**	0,2	2,3
ACO AS	8,4	15,1	19,9	4,4	6,8	7,0
ACO ACS	2,4	10,3	12,5	0,4	2,3	3,3
ACO MMAS	38,3	109,7	165,9	0,4	1,7	1,7
ACO MMAS + 2-opt	**0,1**	**0,2**	5,2	**0,1**	**0,0**	**0,3**

Moyenne sur 3 graines ; un algorithme compte si sa solution est obtenue avant le budget (+10 %).

Annexe : eil51



Annexe : reproduire les résultats

Code : `github.com/BalboniIvanoWacheQuentin/using-ACO-for-TSP`

- `python accuracy_time.py` : complexité et précision vs temps (`results/complexity.csv`, `anytime.csv`)
- `python plot_accuracy_time.py` : figures et tableau par budget
- `python experiments.py` : étude des paramètres de l'ACO
- `python slides/make_figures.py` : figures complémentaires
- `streamlit run streamlit.py` : démonstration interactive (aussi en ligne : `using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`)